

안녕하세요!

저는 **백엔드 개발자 천창현**입니다.

> 저랑 대화해 보실래요? _

CHEON CHANG HYEON



백엔드 개발자 **천창현**

B 1996.08.19
M rearleg96@gmail.com
G www.github.com/rearleg
S www.rearleg.com

About

“커피챗 할래요?”
하루 종일 개발 이야기만 할 수 있습니다.

저는 크리에이터이고, 가장 좋아하는 것은 재귀함수, 개발 그리고 사람입니다.
Java와 Spring으로 백엔드를 개발하고 있고 Go나 Rust를 이용해서 MSA에서 고성능 서버로 최적화하는 기술을 실험 중입니다.
특히 최근에는 Kubernetes를 공부하면서 안정적으로 Scale-out하는 기술에 관심이 생겼습니다. 그 때문에 EC2와 홈 서버를 노드로 두고
여러 상황을 테스트해 보고 있습니다만, 리소스의 제한 때문에 한계가 있습니다. 취직하게 된다면, 더 많은 트래픽 환경에서... [더 보기](#)

... 이제 그만!

Education

2026 [삼성 SW·AI 아카데미(SSAFY)]
15기 서울 지역 대표 / 기자단

2025 [Upstage AI Lab] 6기 수료

2022 [창업부트캠프 창] 6기 수료

2021 [전남대학교] 음악학과 졸업

Awards

2026 [SSAFY 2학기 1차 프로젝트]
우수상 (1등)

[SSAFY 1학기 최종 프로젝트]
최우수상 (1등)

[SSAFY 성적 우수상]

[공공데이터 활용 섬박람회 아이디어 공모전]
여주시 / 최우수상 (1등)

Experience

2025 [스타트업 창업팀] ioBro
AI 모델 개발

2020 [산학 협력 프로젝트]
대학생 공연/전시 'The Color' 기획

2019 [영상 프로덕션 운영] 스튜디오 친칠라
대기업 및 공공기관 영상 제작

2018 [영화 제작 동아리]
중앙동아리 설립 및 1~3기 회장

2017 [CJ 도너스 캠프]
대중음악 부문 멘토

개발은 즐거워, 늘 새로워!

서비스에 필요한 기술을 빠르게 배워 프로젝트에 적용합니다.

Languages



Java 80%

객체지향 프로그래밍에 능숙하며,
대다수의 프로젝트에 적용했습니다.



Python 80%

AI 파이프라인 코드 작성이 가능하며,
uv를 활용한 환경 관리에 능숙합니다.



Go 40%

기본적인 코드 작성과 goroutine을
활용한 경험이 있습니다.



Rust 40%

언어의 특성을 이해하고 있으며,
기본적인 문법과 코드 작성 경험이 있습니다.



JS / TS 40%

기본 문법을 어느 정도 숙지하고 있습니다.
간단한 페이지 작성 경험이 있습니다.

Frameworks



Spring 80%

MVC, Security, JPA 등을 이용하여
대다수의 프로젝트에 적용했습니다.



Django 60%

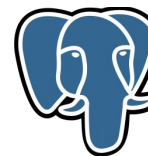
Django를 통해 OOP에 입문하였으며,
CRUD와 인증 로직을 작성할 수 있습니다.



PyTorch 60%

영상/음성 도메인에서 전처리부터
학습, 평가 코드까지 작성할 수 있습니다.

DB



PostgreSQL 60%

쿼리 최적화와 인덱스 설계 및
pgvector 검색을 구현할 수 있습니다.



Redis 40%

Cache-Aside 패턴을 적용하고,
TTL 캐시 정책을 구성한 경험이 있습니다.

Infra / DevOps



Docker 80%

캐시 마운트, 이미지 최적화가 가능하며,
대다수의 프로젝트에 사용하였습니다.



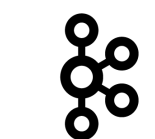
AWS 60%

애플리케이션 배포 및 클라우드 인프라를
구성하고 운영할 수 있습니다.



Nginx 60%

리버스 프록시와 로드밸런싱을 구성하고,
애플리케이션 트래픽을 관리할 수 있습니다.



Kafka 40%

Producer/Consumer를 구성하고,
비동기 메시징을 구현해 본 경험이 있습니다.



Jenkins 40%

Jenkins Pipeline을 활용해,
CI/CD를 구성해 본 경험이 있습니다.

Monitoring



Prometheus 40%

메트릭을 수집하고 모니터링 환경을
구성한 경험이 있습니다.



Grafana 40%

메트릭을 시각화하고,
모니터링 프로토타입 구성 경험이 있습니다.

OS / IDE / Collaboration



PROJECTS

Project 1

AR 원격 하자 검수로
임대차 분쟁을 해결하는 월세 관리 플랫폼

방긋

[우수상(1등) 수상]

서버 자원을 고려한 최적화한 프로젝트입니다.
MSA와 Kafka가 능사가 아니라는 것을
배웠습니다.

5p

Project 2

AI 퍼스널 컬러 진단을 통한
아이템 추천 및 학생증 꾸미기 커머스

hakku

[최우수상(1등) 수상]

MSA 구축과 성능 모니터링으로
이미지 입출력 서버의 최적화까지 진행했던
프로젝트입니다.

9p

Project 3

AI 발표 분석으로
말하기 습관을 개선하는 발표 코칭 서비스

selo

[엑셀러레이팅 및 박람회 참가]

비유창성 탐지 모델을 만들고
사용자 테스트까지 진행했습니다.
개발에 입문한 소중한 프로젝트입니다.

13p

and more

AI 주도 개발 방법론을 실험하고
실용적인 미디어아트 작품을 만들었습니다.

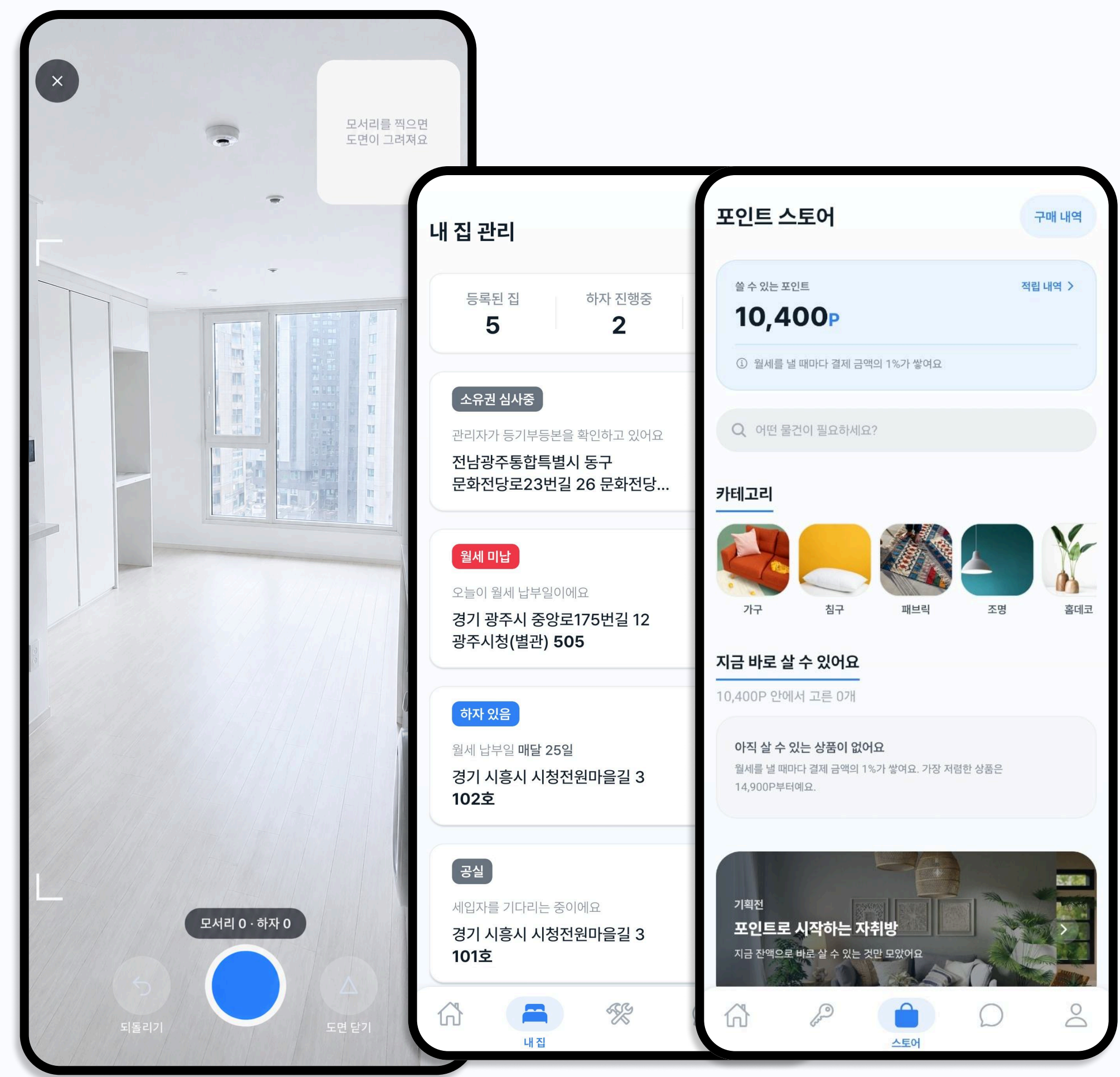
17p

방긋

AR 원격 하자 검수로 임대차 분쟁을 해결하는 월세 관리 플랫폼

개발 편의성과 유지보수성을 고려한
멀티 스테이지 배포, DevOps/MLOps를 적용하여
배포 시간을 80% 이상 단축했습니다.

기간	2026.07 - 2026.08 (6W)
역할	백엔드/인프라 개발, 팀장
구성	백엔드/인프라 개발(1), AI 개발(1), 프론트엔드 개발(2), 백엔드 개발(2)
기여도	30%
성과	[SSAFY 2학기 1차 프로젝트 우수상(1등) 수상] - Scale-out을 고려한 최적화 - 사용할수록 성능이 개선되는 MLOps 구축 - 리소스 한계를 극복하는 메모리 최적화
GitHub	github.com/a205-banggoot/banggoot-public
Live	www.banggoot.com

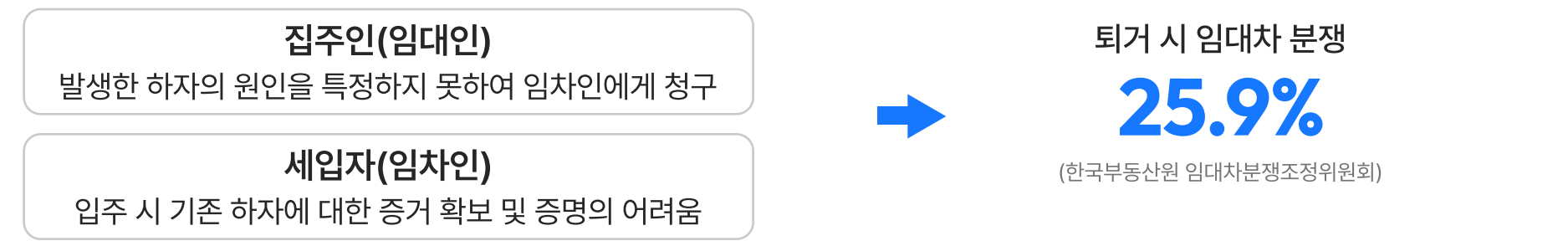


1. 프로젝트 개요

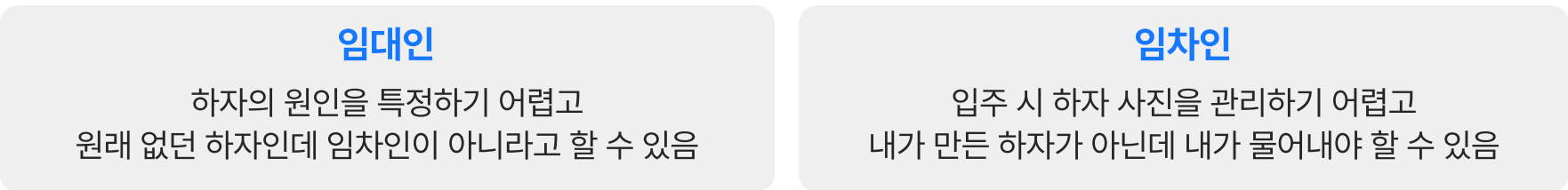
임대인과 임차인이 **실시간으로 도면을 생성**하고 **하자를 함께 기록**하여,
퇴거 시 원상복구 분쟁을 예방하는 **임대차 관리 서비스**

1) 문제 인식

임차인의 퇴거 시 하자 복구에 대한 **책임 소재가 불분명**하여 **임대차 분쟁**이 생깁니다.



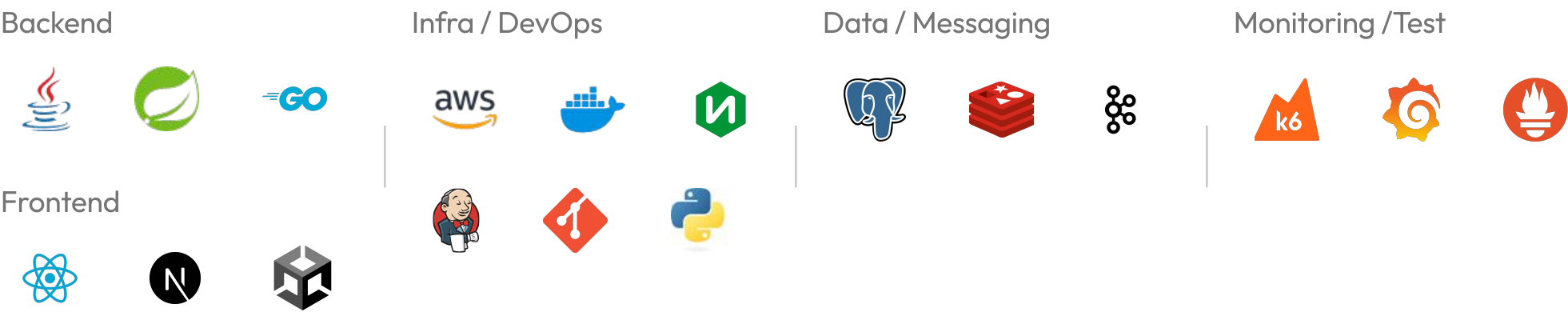
Pain Point



2) 담당 역할



3) 사용 기술



2. 핵심 기능

실시간 원격 하자 검수

집주인과 실시간으로 집을 보며 하자를 확인하고 서로 합의하는 과정을 거칩니다.
1) AR 도면 제작 → 2) 하자 사진 촬영 → 3) AI 하자 분류 순으로 진행됩니다.

1) AR 도면 제작

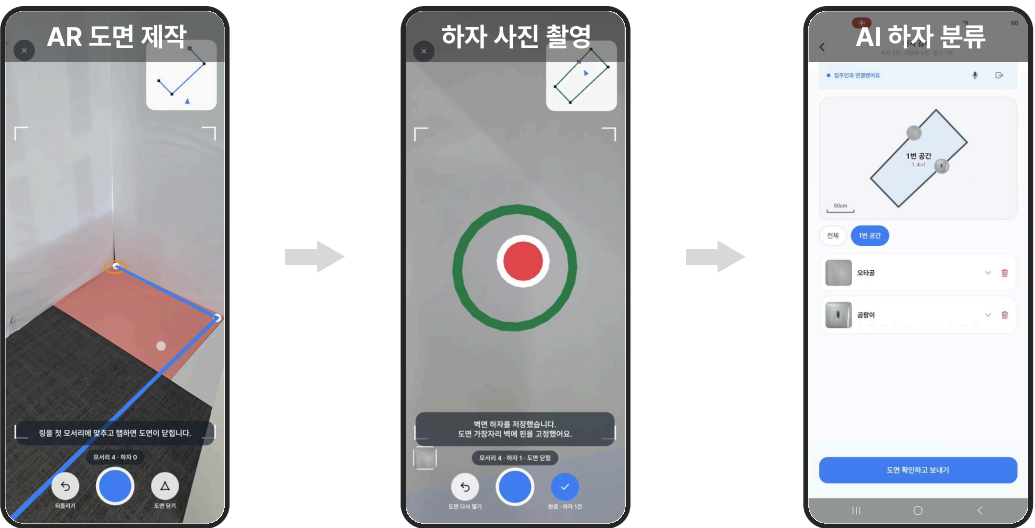
하자 위치를 기록하고 관리하기 위해
모서리를 기록하여 방 안의 도면을 제작합니다.

2) 하자 사진 촬영

하자 부분을 확인하고 사진을 찍으면
제작한 도면에 정확한 위치와 함께 사진이 기록됩니다.

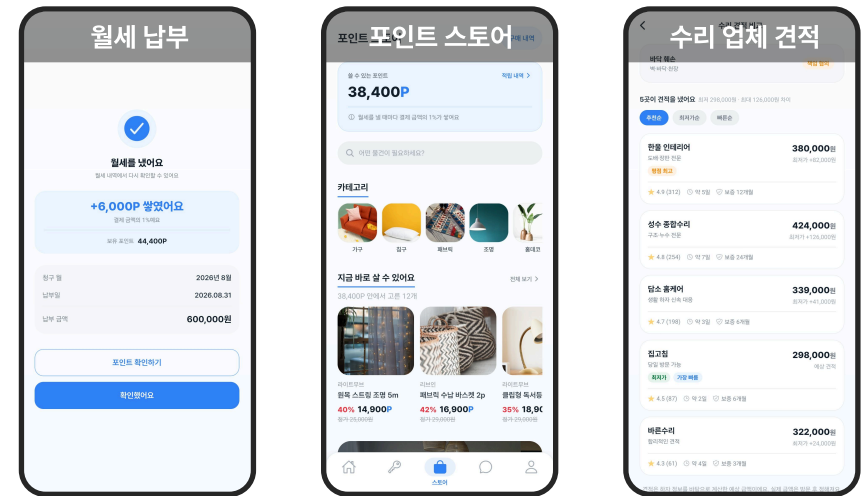
3) AI 하자 분류

촬영된 하자 사진을 보고 AI가 종류를 분류합니다.
분류된 사진과 하자를 서버에 기록합니다.



세입자 - 월세 포인트 현금 및 포인트 스토어

앱 내에서 월세를 납부할 수 있으며 포인트를 받아,
생활 중에 생긴 하자를 보수하거나, 인테리어 소품을 구매하는 데 사용할 수 있습니다.



월세 납부 및 포인트 현금

월세나 관리비를 납부하고 포인트를 현금받을 수 있습니다.

포인트 스토어

포인트를 모아 생필품이나 인테리어 소품을 구입할 수 있습니다.

수리 업체 견적

수리 업체에 견적을 받아 손쉽게 집을 보수할 수 있습니다.

집주인 - 임대 관리 대시보드

앱과 웹에서 여러 집을 한 페이지에서 쉽게 관리할 수 있습니다.

관리 대시보드

월세, 관리비 등 여러 주택의 임대 정보를 한곳에서
관리할 수 있습니다.

하자 관리 및 수리

하자를 세입자와 협의하고 책임 소재를
명확히 할 수 있습니다.



3. 트러블 슈팅

1) 이미지 크기와 배포 시간으로 인한 Scale-out 제약

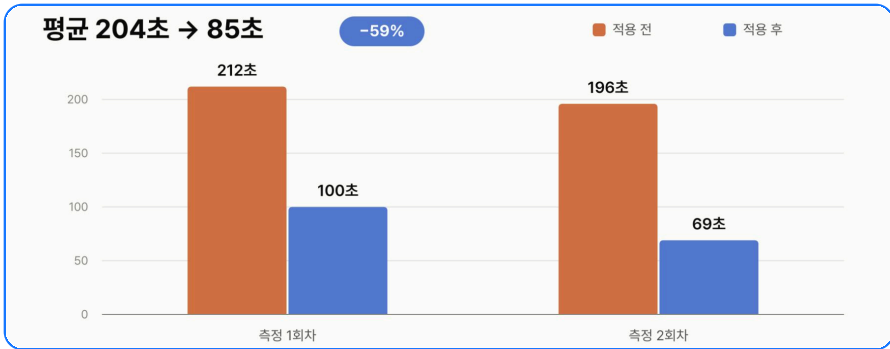
상황

월세 관리와 하자 검수는 기능별 **사용 빈도와 트래픽 패턴이 달랐기 때문에**, 자원 효율화를 위해 서비스별 독립적인 **Scale-out이 가능하도록 설계**했습니다. 하지만 큰 용량의 컨테이너와 긴 배포 시간은 **트래픽 증가에 빠르게 대응하기 어렵게 만들**었습니다.

해결

a. 캐시 마운트 적용으로 204초 → 85초 (58.3% 감소)

```
RUN --mount=type=cache,target=/root/.gradle,sharing=locked \
./gradlew :${SERVICE}:bootJar --no-daemon -x test
```



b. 멀티 스테이지 배포로 디스크 용량을 858MiB → 559MiB (35% 감소)

빌드와 런타임 스테이지를 분리하여 컴파일된 JAR 파일만 포함해 공간을 절약했습니다.



회고

가용 리소스를 고려한 확장성 설계

Scale-out이 가능한 구조를 만들 때, 새 인스턴스를 **빠르고 가볍게** 추가할 수 있도록 하는 게 **중요**하다는 것을 배웠습니다. 제한된 리소스 환경에서 이미지 크기와 배포 시간이 **확장성의 병목**이 될 수 있다는 것을 경험했습니다. 이후에는 **컨테이너 레지스트리를 도입**해 빌드된 이미지를 재사용함으로써 배포 시간을 더욱 단축해볼 계획입니다.

2) 부족한 데이터로 하자 분류 모델 성능 저하

상황

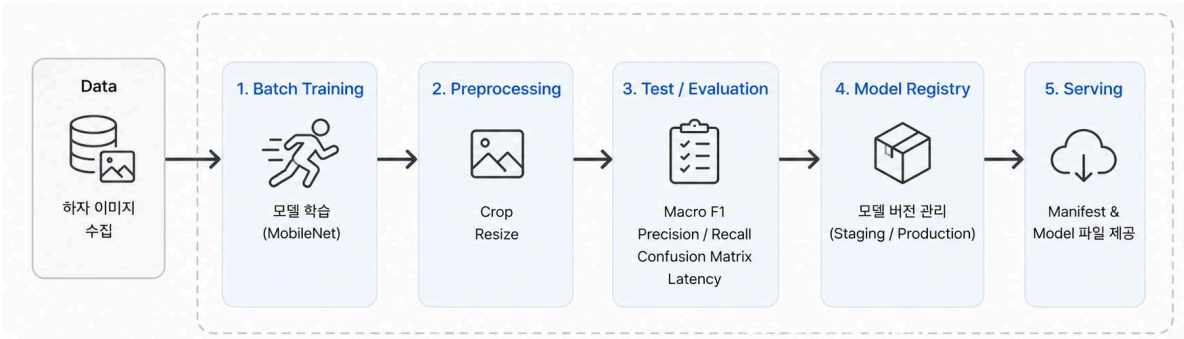
실시간 통신 환경에서 발생하는 네트워크 오버헤드와 지연을 줄이기 위해, **온디바이스 추론을 적용**했습니다. 하지만 프로젝트 기간 내 **충분한 데이터를 확보할 수 없었**습니다. 이러한 상황에서 **모델 성능을 개선할 방법**을 찾아야 했습니다.

해결

a. 사용자 하자 데이터 수집 → 학습 → 배포 자동화 MLOps 구축

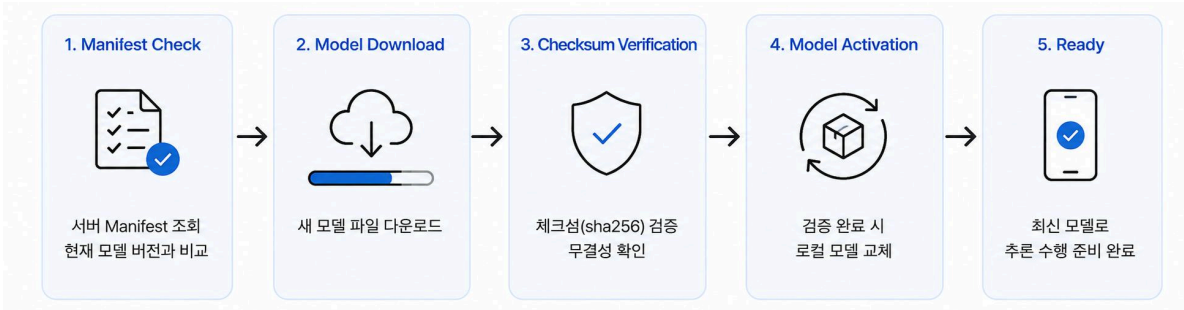
“데이터가 부족하면 사용할수록 모델이 개선되게 만들자”

서비스 사용 과정에서 생성되는 하자 데이터를 수집하고 재학습하는 구조를 설계했습니다.



b. 앱 실행 시 Model Sync - Client

앱 실행 → 매니페스트 체크 → 모델 업데이트로 클라이언트에서 사용하는 모델을 최신 버전으로 유지합니다.



회고

성능 개선을 위한 새로운 접근법 터득

시스템이 **지속적으로 개선될 수 있는 구조**를 만들었습니다. 이 경험을 통해 점진적으로 해결되도록 시스템을 설계하는 것도 **중요한 문제 해결 방식**이라는 점을 배웠습니다. 향후에는 AIOps를 통해 모델 학습과 CI/CD 등 전체 **오퍼레이션을 통합하는 시스템을 설계**하고자 합니다. 이후 기존 방식과 성능을 비교하며 AI로 제어하는 워크플로우 구축까지 이어갈 계획입니다.

3) 프로젝트 종료 후 서버 메모리 문제

상황

프로젝트 개발 서버(t3.xlarge/16GB)의 운영이 종료되면서 배포를 위해 개인 서버(t3.small/2GB)로 옮겨야 했습니다만, 기존 **MSA 스택이 약 3,853 MiB**를 차지하고 있어 서버가 감당하기 어려워 **OOM이 발생**했습니다.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
a3ce8d57f4ef	banggoot-msa-local-mlops-1	1.37%	392MiB / 512MiB	76.55%	111kB / 133kB	107MB / 111kB	37
4b60f3f33f0b	banggoot-msa-local-storage-server-1	0.00%	8.035MiB / 256MiB	3.14%	3.54kB / 126B	17.7MB / 0B	8
8f56f0bcb511	banggoot-msa-local-chat-1	2.03%	371.9MiB / 384MiB	96.73%	193kB / 236kB	163MB / 10.1MB	62
4d5a5cf18958	banggoot-msa-local-property-1	0.72%	398.7MiB / 512MiB	77.87%	130kB / 131kB	93.8MB / 115kB	37
6394551c24ff	banggoot-msa-local-identity-1	1.89%	341.5MiB / 384MiB	88.93%	106kB / 123kB	218MB / 102kB	36
705aefcf81baa	banggoot-msa-local-analysis-1	0.63%	340.8MiB / 384MiB	88.75%	138kB / 171kB	105MB / 106kB	37
d3d3ef782194	banggoot-msa-local-notification-1	1.70%	358.4MiB / 384MiB	93.34%	105kB / 124kB	90.9MB / 102kB	36
982954c80e3c	banggoot-msa-local-inspection-1	1.39%	423.2MiB / 640MiB	66.12%	220kB / 208kB	96.4MB / 115kB	43
7bc00941a057	banggoot-svc-db	0.41%	139.1MiB / 31.1GiB	0.44%	632kB / 559kB	8.72MB / 307kB	76
cd122b5a9a53	banggoot-svc-redis	3.07%	6.051MiB / 31.1GiB	0.02%	106kB / 60.8kB	7.3MB / 0B	6
14b2723cf7f9	banggoot-kafka-kafka-1	4.33%	507.9MiB / 640MiB	79.36%	396kB / 359kB	530MB / 202MB	116
6d5581d96d36	banggoot-local-backend-1	1.50%	443.6MiB / 1GiB	43.32%	288kB / 200kB	84.1MB / 217kB	57
59df36accfce	banggoot-local-postgres-1	2.55%	60.66MiB / 512MiB	11.85%	124kB / 235kB	39.2MB / 315kB	18
82a96805054	banggoot-local-redis-1	2.21%	9.445MiB / 256MiB	3.69%	80kB / 51.3kB	23.7MB / 4.1kB	12

해결

a. 런타임 오버헤드 제거를 위한 모놀리식 전환

모놀리식으로 변경 후 3,853 MiB → 395 MiB로 **약 90% 감소**

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
b0959564bd61	banggoot-mono-app	0.18%	259.4MiB / 896MiB	28.95%	13.7GB / 3.31GB	2.32GB / 2.59GB	42
6444f3212c23	banggoot-mono-rtc	0.00%	4.254MiB / 96MiB	4.43%	0B / 0B	59.7MB / 10.4MB	8
a3b70637c910	banggoot-mono-web	0.00%	68.61MiB / 128MiB	53.60%	12.7MB / 122MB	631MB / 213MB	11
91b9f6faf6ea	banggoot-mono-postgres	0.08%	31.75MiB / 256MiB	12.40%	3.68GB / 16.6GB	574MB / 215MB	10
f3cdb7ace007	banggoot-mono-redis	0.57%	3.648MiB / 80MiB	4.56%	349MB / 205MB	880MB / 878MB	6
b5f23ea1bc75	banggoot-mono-coturn	0.02%	1.199MiB / 96MiB	1.25%	0B / 0B	25.9MB / 7.37MB	7
3975e4e6327b	banggoot-mono-storage	0.00%	5.598MiB / 64MiB	8.75%	38.9kB / 126B	145MB / 10.1MB	8

b. 컨테이너 유희 메모리 공유

그럼에도 요청이 생기거나 메모리 사용량이 급증하면 **OOM이 지속적으로 발생**하여, 컨테이너 **메모리 오버커밋 적용**하여 개별 합 1,616 MiB → slice **1,000 MiB로 제한**

```
19: app:
26:   cgroup_parent: banggoot.slice
29:   mem_limit: 896m
61:   postgres:
64:     cgroup_parent: banggoot.slice
68:     mem_limit: 256m
98:   redis:
99:     cgroup_parent: banggoot.slice
107:     mem_limit: 80m
109:   storage-server:
114:     cgroup_parent: banggoot.slice
131:     mem_limit: 64m
134:   web:
143:     cgroup_parent: banggoot.slice
155:     mem_limit: 128m
168:   rtc-server:
164:     cgroup_parent: banggoot.slice

➤ cat /etc/systemd/system/banggoot.slice
[Unit]
Description=banggoot stack memory cap
[Slice]
MemoryAccounting=yes
MemoryHigh=900M
MemoryMax=1000M
```

회고

MSA와 Kafka가 능사가 아니다.

메모리 최적화를 통해 아키텍처는 **실제 운영 가능한 리소스와 비용까지 고려해 설계**해야 한다는 점을 배웠습니다. 이후에는 서비스별 메모리 사용량과 **리소스 한도를 검토**하고, 제한된 환경에서도 **안정적으로 운영**할 수 있는 구조를 설계하고자 합니다.

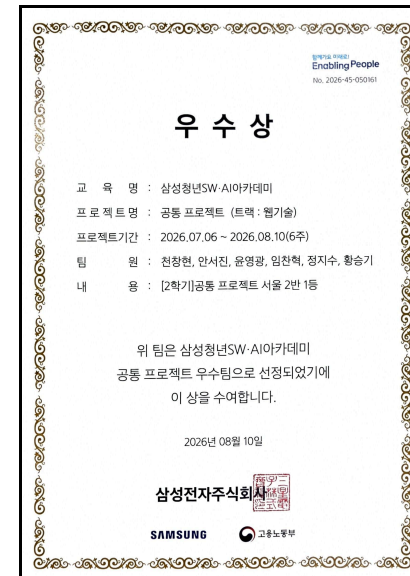
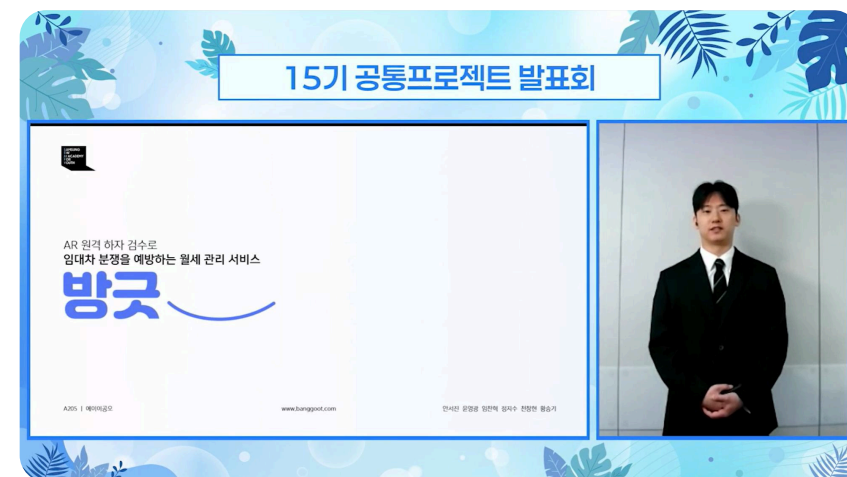
삼성SW·AI아카데미 2학기 1차 프로젝트

우수 프로젝트 1등 선정

성과

15기 공통 프로젝트 우수상(1등) 수상
(공통 프로젝트 : 2학기 1차 프로젝트)

결선 발표회 | 상장



산출물



Web



App



GitHub

공부한 것

WebRTC - Go로 WebRTC 서버 만들기
WebRTC 개념, SFU, TURN, ICE

DevOps - Jenkins를 이용한 CI/CD 구현
CI/CD 파이프라인, Jenkinsfile, Docker 배포

MLOps - 데이터 수집, 학습, 배포 자동화 파이프라인
데이터 파이프라인, 학습 자동화, 모델 배포

Scale-out을 위한 아키텍처 설계
컨테이너 캐시 마운트, 멀티 스테이지 배포, 레지스트리

Docker 최적화
오버커밋, Slice를 적용하여 메모리 사용량 제한

AI 활용

BMAD Method(기획)
기획, 설계, 문서화 생성 워크플로우 사용

ECC 워크플로우
Plan - TDD - Code Review 구현 워크플로우 사용

온디바이스 모델 - MobileNet + ONNX(양자화)
모델 양자화 및 ONNX Runtime, NPU 추론 최적화

프로젝트 회고 및 인사이트

- 초기에 MSA와 Kafka를 도입하며 시스템을 구동할 리소스가 부족 문제 발생
- 이를 해결하는 과정에서 컨테이너 최적화를 경험
- 기술적 이상보다 **자원과 운영 여건**을 함께 고려해 **아키텍처를 선택**해야 한다는 것을 학습

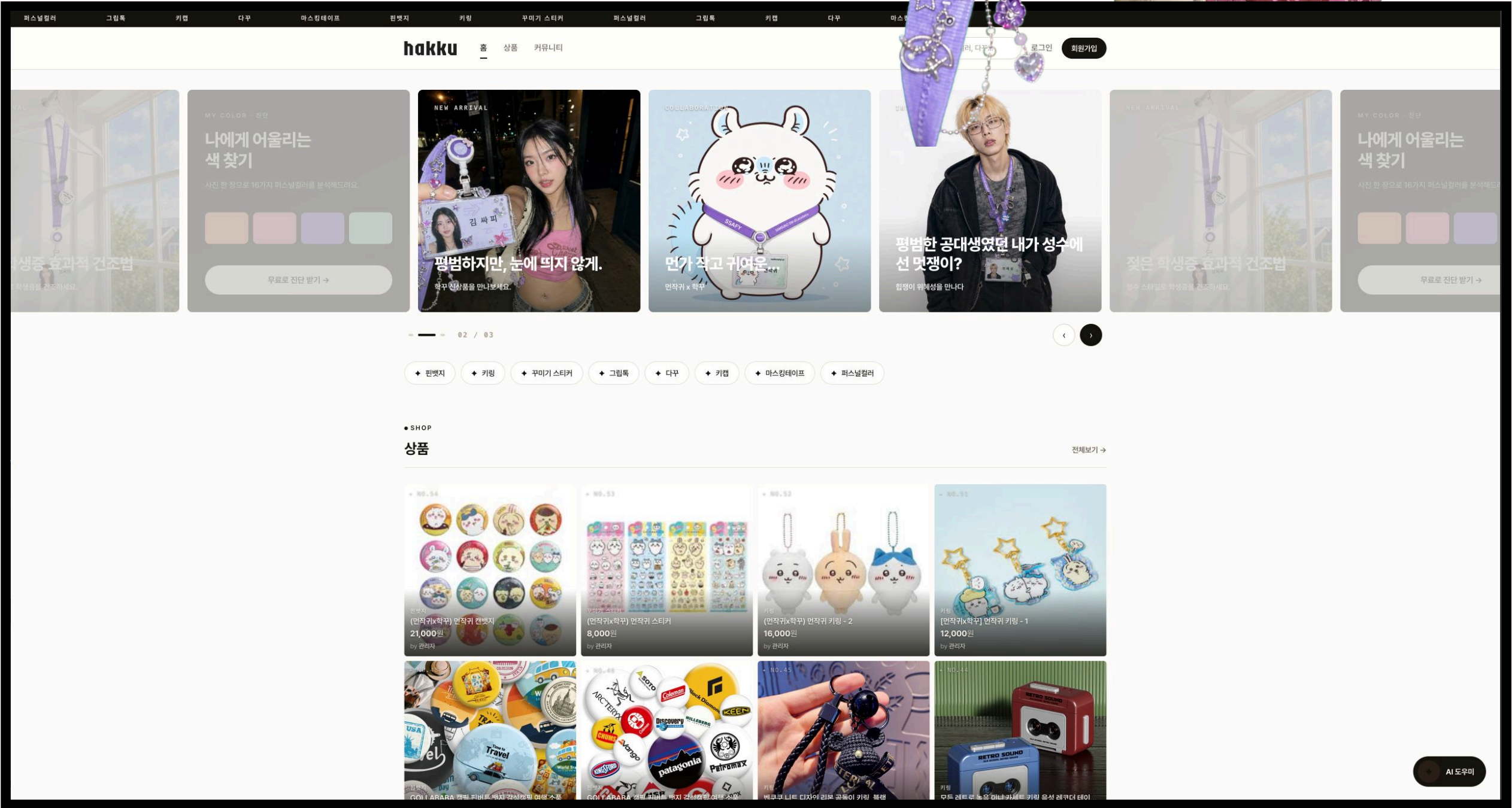


AI 퍼스널 컬러 진단을 통한 아이템 추천 및 학생증 꾸미기 커머스

마이크로서비스 아키텍처로 설계하여 AI 추론을 비동기 처리하고
Go를 통해 이미지 서버 초당 처리량을
287% 개선했습니다.

기간	2026.06 - 2026.07 (5W)
역할	백엔드/인프라 개발, 팀장
구성	백엔드/인프라 개발(1), 프론트엔드 개발(1)
기여도	70%
성과	[SSAFY 1학기 최종 프로젝트 최우수상(1등)] [SSAFY 1학기 성적 우수상] - AI 추론 비동기 아키텍처 구현 - 성능 모니터링 및 벤치마킹 구현 - Go 최적화로 초당 처리량 287% 향상

GitHub	github.com/hakku-ssafy/hakku
Live	hakku.rearleg.com



1. 프로젝트 개요

나의 사진을 올리면 **AI가 퍼스널 컬러를 분석**하고,
나에게 어울리는 **꾸미기 아이템**을 추천해주는 커머스 서비스

1) 개발 배경

얼굴과 가장 가까운 곳, 목에 거는 **사원증의 색이 인상을 좌우**합니다. 하지만,
나와 **잘 맞는 적절한 꾸미기 아이템**을 구매하는 데에 **어려움**이 있습니다.

어떤 색이 나에게 잘 맞는지 모르겠어요

어디서 사야 할지 모르겠어요

비싼 퍼스널 컬러 분석

100,000 ~ 200,000원

파편화된 쇼핑 채널



미국 간호사 **badge reel** 꾸미기 유행 → 직장인 엑세서리로 **자기표현의 시장성**을 확인했습니다.



NURSE.COM BLOG
Choosing the Right Badge Reel for Your Nursing Needs

숏폼 영상을 통해 꾸미기 사례를 소개

적절한 배지 릴 선택에 대한 가이드 (nurse.com)

2) 담당 역할

MSA
아키텍처 설계

보안 및 안정성
JWT 인증 / Owner 검증

인프라
홈 서버 온프레미스 구축

성능 모니터링
k6 / Prometheus 관측

3) 사용 기술

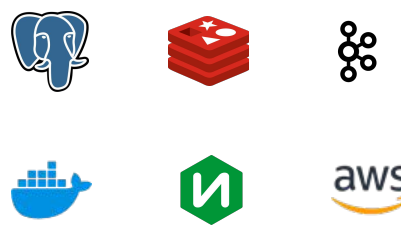
Backend



Frontend



Data / Infra



Monitoring / Test



2. 핵심 기능

AI 퍼스널 컬러 진단

얼굴이 잘 나온 사진을 업로드하면, AI가 사진을 전처리한 후 퍼스널 컬러를 진단합니다.



퍼스널 컬러 진단 온보딩

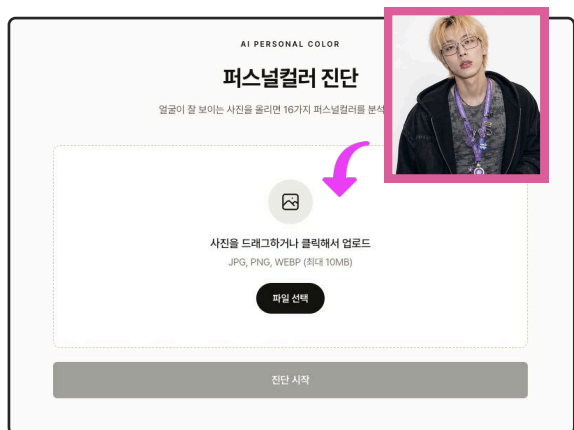
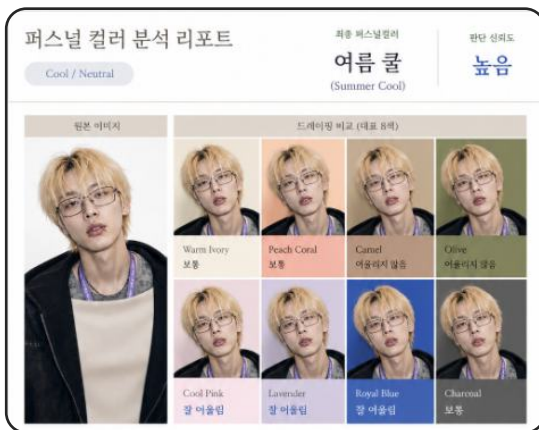


사진 업로드



퍼스널 컬러 진단 결과지

퍼스널 컬러 + 선호 컬러 기반 상품 추천

내게 맞고, 내가 좋아할 만한 꾸미기 아이템을 추천해 줍니다.

온보딩 - 선호 컬러 조사

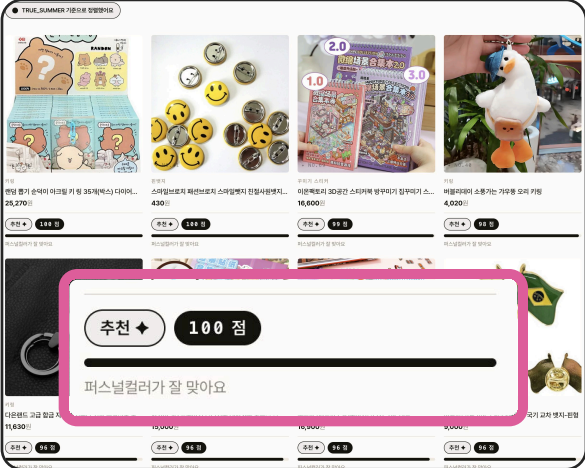
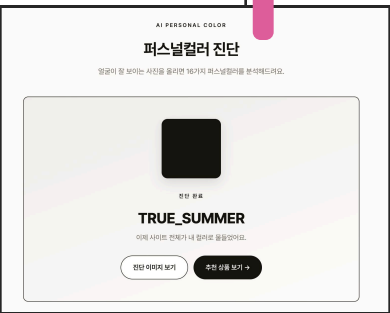
회원가입 시 좋아하는 컬러를 선택합니다.

퍼스널 컬러 진단

퍼스널 컬러를 진단받습니다.

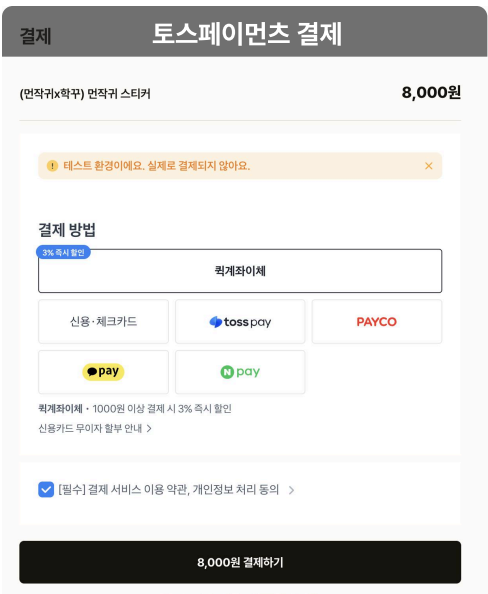
인기 상품

구매, 좋아요, 후기 등으로
상품의 가중치를 더합니다.



커머스 & 커뮤니티

꾸미기 아이템을 구매하고 꾸민 사원증을 자랑하거나 정보를 공유할 수 있습니다.



3. 트러블 슈팅

1) 분석 요청 후 5분간 사용자가 페이지를 못 떠나는 문제

상황

퍼스널 컬러 진단이 **한 요청에 약 5분**이 걸렸습니다. 그동안 사용자는 분석이 끝날 때까지 해당 페이지에 머물러야 했고, **나가거나 새로고침하면 처음부터 다시 진행**해야 하는 상황이었습니다.

해결

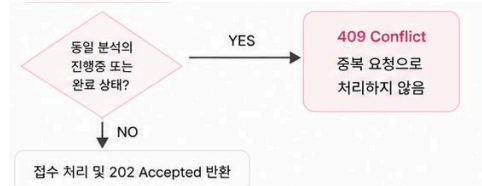
a. 요청 접수와 처리를 분리하여 비동기 처리 구현



요청 상태 단계



중복 요청 처리



b. Kafka와 Redis를 이용하여 완료 알림 구현

사용자가 페이지에서 벗어나거나, 로그아웃해도 돌아와서 확인할 수 있습니다.



회고

비동기 처리로 사용자 경험과 안정성을 개선

긴 분석 작업을 요청 접수와 분리하면서, 사용자가 페이지를 떠나도 작업이 유지되는 구조를 만들었습니다. 안정적인 비동기 처리는 **상태 관리**와 **중복 요청 방지**까지 함께 고려해야 한다는 점을 배웠습니다.

2) 이미지 로딩 지연 문제

상황

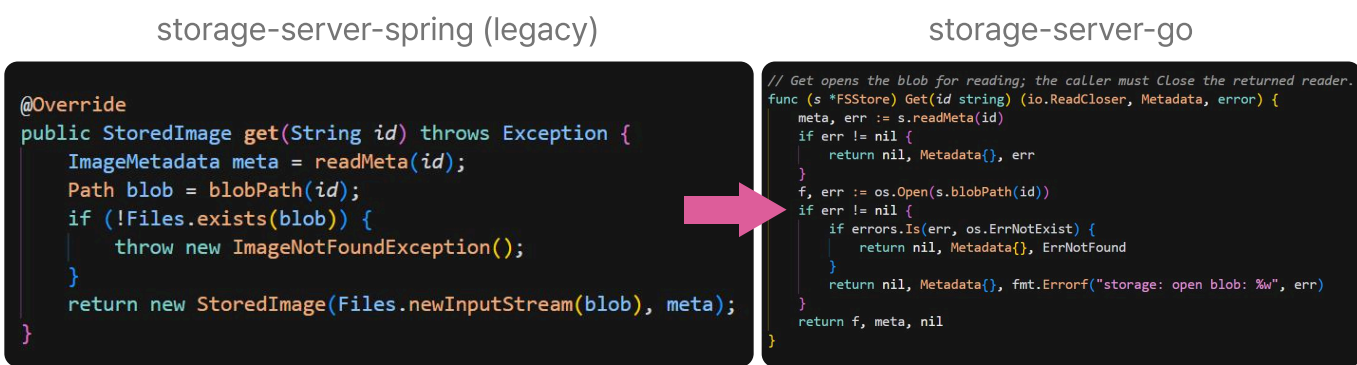
이미지 위주의 서비스인데 **업로드와 조회가 눈에 띄게 느렸습니다**. 이미지는 Storage Server에 저장되는데, 해당 서버는 **비즈니스 로직보다 I/O 작업이 주가 된다**는 점에 주목했습니다.



해결

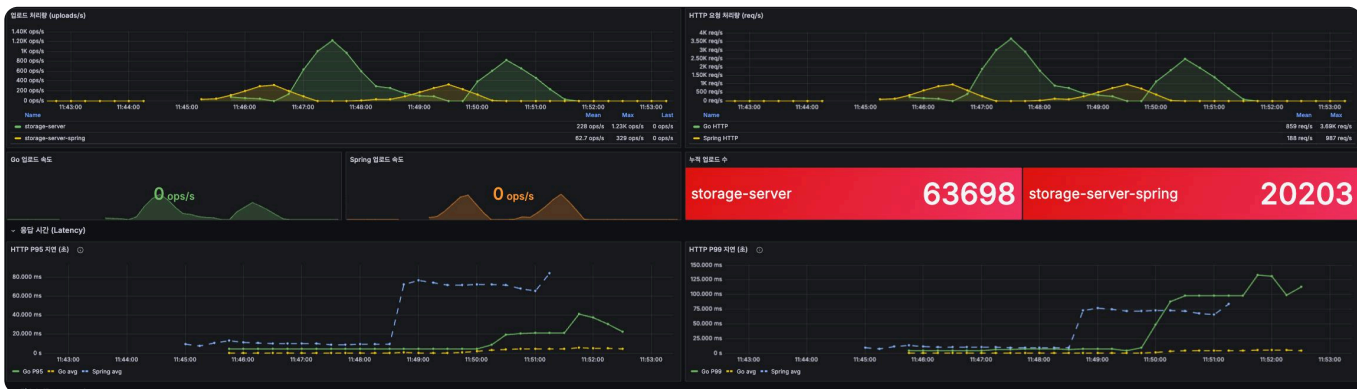
a. I/O 작업의 비중이 높아 Spring → Go 변경

가벼운 런타임과 높은 I/O 동시 처리 성능을 가진 Go로 마이그레이션



b. Storage Server 벤치마크 결과 : 처리량 287% 개선

k6를 통해 이미지 I/O 벤치마크를 수행했고, Go 변경이 효과가 있음을 증명했습니다.



회고

적절한 기술 선택의 중요성

JVM과 무거운 프레임워크보다 가벼운 런타임을 갖는 Go로 마이그레이션하고 부하테스트를 통해 **수치적으로 검증**하는 경험을 했습니다. 이를 통해 서비스 **특성에 맞는 기술 선택**과 **데이터 검증의 중요성**을 배웠습니다.

3) 사용자 업로드 사진의 용량이 큰 문제

상황

스토리지 서버를 최적화해도 사용자가 올리는 이미지가 크면 문제가 반복될 수 있다고 판단했습니다. 업로드된 사진 **용량에 비해** 화면에 표시되는 **사진 크기가 매우 작다**는 걸 발견했습니다.

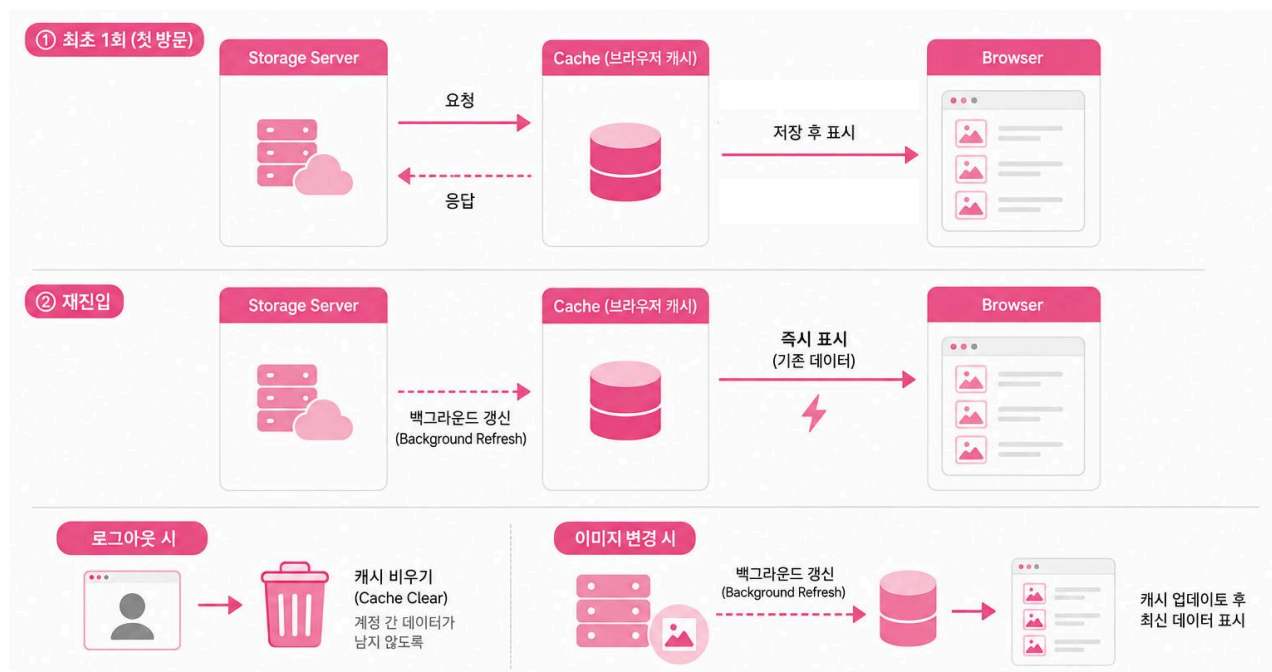
해결

a. 업로드 전 전처리 로직 추가로 4MB → 350KB (약 91% 절감)

Canvas로 1MB가 넘는 이미지는 긴 변 1920px, WebP 변환을 수행합니다.



b. 목록 조회에 캐시 적용



회고

성능 최적화를 위한 관점 확장

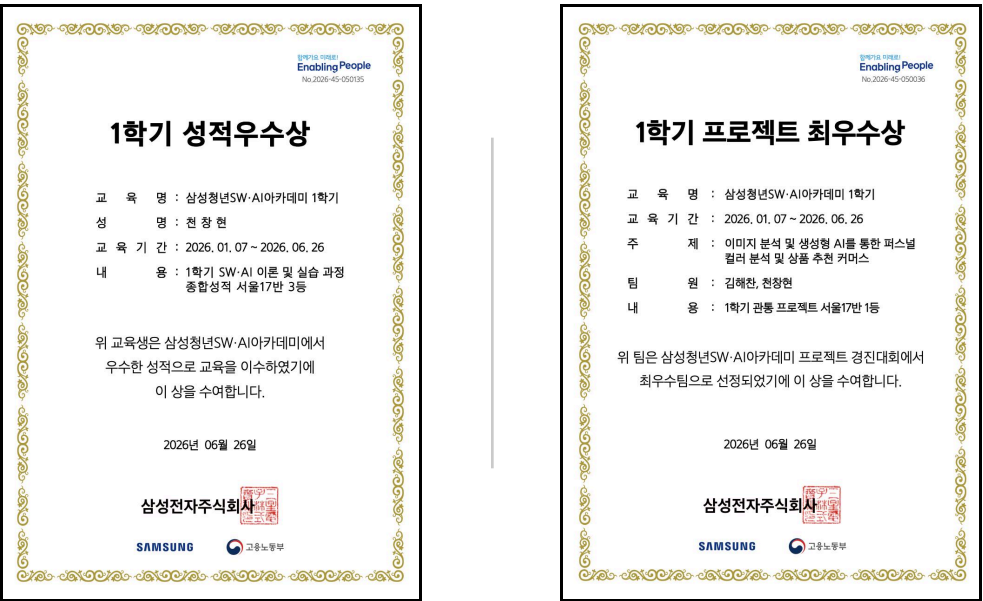
클라이언트가 어떤 형태와 크기로 데이터를 전달하는지가 **성능에 큰 영향**을 준다는 것을 배웠습니다. 이를 통해 최적화를 전체 데이터 흐름의 관점에서 평가하고 **다양한 개선 방법**을 고민하는 계기가 되었습니다.

삼성SW·AI아카데미 1학기 최종 프로젝트 최우수 프로젝트 1등 선정

성과

15기 관통 프로젝트 최우수상(1등) 수상
(관통 프로젝트 : 1학기 최종 프로젝트)

성적우수상 | 상장



산출물



Web



GitHub

공부한 것

- MSA/Kafka**
기능별 결합도 완화 및 비동기 메시징 처리
- Redis**
인메모리 캐싱과 Cache-Aside 패턴 학습
- Go**
기본 문법과 Goroutine 비동기 처리 학습

AI 활용

- chatGPT Image 2**
이미지 생성에 text-to-image 모델 사용
- superpowers 스킬**
brainstorming 등 기획 문서 제작에 사용
- ECC 워크플로우**
Plan - TDD - Code Review 구현 워크플로우 사용

프로젝트 회고 및 인사이트

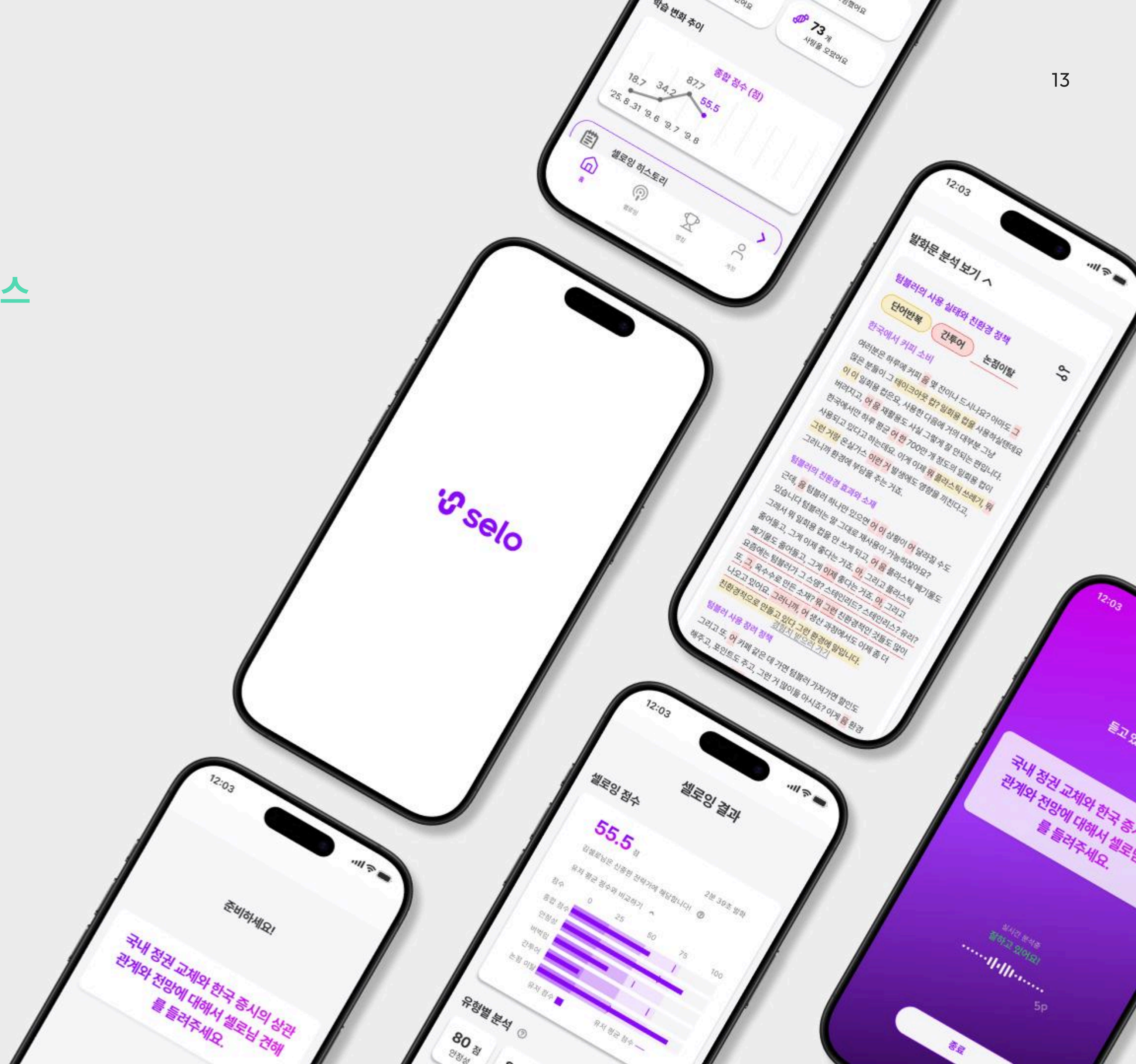
- 성능 모니터링을 도입해 병목 지점을 특정
- 여러 가설이 하나의 사실로 좁혀지는 과정을 경험하며, **관측 시스템의 중요성**을 깨달음
- 서비스별로 서로 다른 언어를 적용해보며, 각 서비스의 **특성에 맞는** 기술을 선택하는 것이 **하나의 최적화 방법**이 될 수 있다는 점을 학습



AI 발표 분석으로 발표자의 말하기 습관을 개선하는 발표 코칭 서비스

비유창성 탐지 모델 SeloWhisper를 만들어
유헤 자원을 이용해 GPU 추론 서버에 배포하고
사용자 테스트까지 진행했습니다.

기간	2025.05 - 2025.10
역할	백엔드, AI, 팀장
구성	백엔드/AI(1), 프론트엔드(1), PM(1), 사업관리(1)
기여도	50%
성과	[엑셀러레이팅 사업 선정] - 비유창성 탐지 모델 SeloWhisper 토큰 f1-score 0.92 달성 - AI 추론용 온프레미스 서버 구축 - 배포 및 사용자 테스트 진행 (73명, 만족도 83.6%)



1. 프로젝트 개요

어디서든 발표 연습을 하고

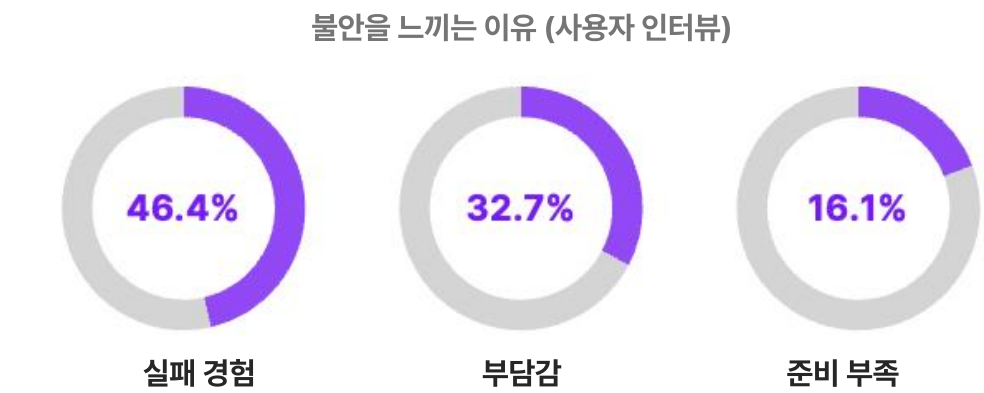
정확한 데이터로 발화 분석 및 피드백을 제공하는 스피치 트레이닝 AI

1) 문제 인식

연구원, 대학원생, (예비)창업가 등 주기적인 발표를 통해 성과를 증명하는 ‘성과 증명 집단’이 존재하지만, 이들의 발표 불안을 해결할 수 있는 서비스는 시장에 부족합니다.



Pain Point



발표로 준비한 것을 보여주지 못하며, 이를 해결하기 위해 혼자 막연하게 연습하거나, 비용이 많이 드는 학원에 다녀야 합니다.

2) 담당 역할

Backend
EC2 기반 API 서버 구축

AI Dev
비유창성 탐지 모델 개발

추론 서버
온프레미스 GPU 서버 구축

팀 운영
프로젝트 기획 및 운영

3) 사용 기술

Backend

Python, Django, FastAPI

Frontend

TypeScript, React, Next.js, Nuxt.js

Data / Infra

PostgreSQL, Redis, AWS, Docker, Kubernetes

AI

PyTorch, TensorFlow, OpenAI, CosyVoice3

2. 핵심 기능

주제 발표

관심 주제나 발표문 스크립트를 통해 AI가 유창성, 톤, 발화 속도를 측정하여 피드백을 제안합니다.

1) 주제 선택

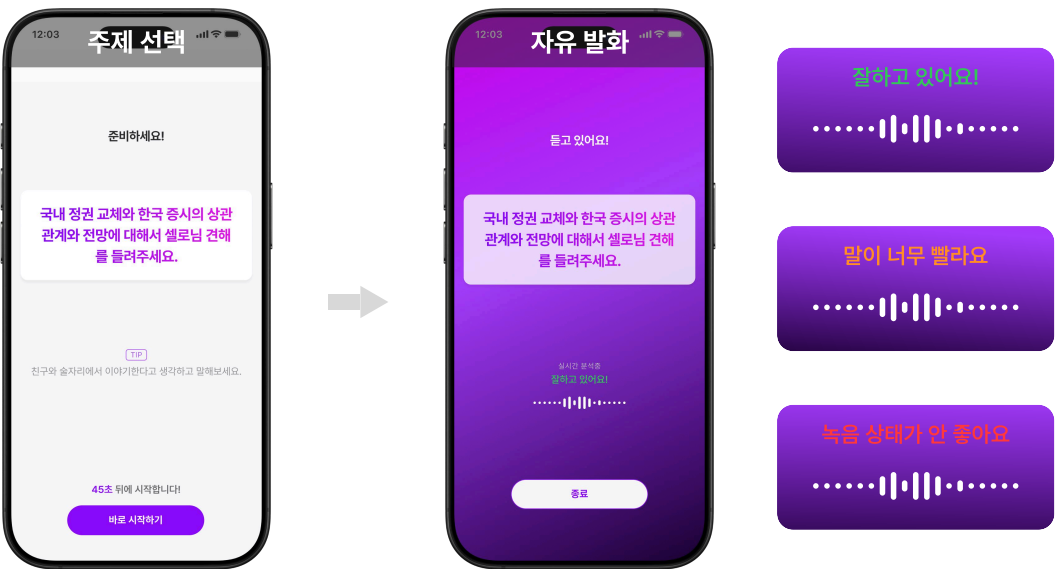
관심사 주제를 선택하거나 발표문 스크립트를 업로드할 수 있습니다.

2) 녹음 및 실시간 발화 모니터링

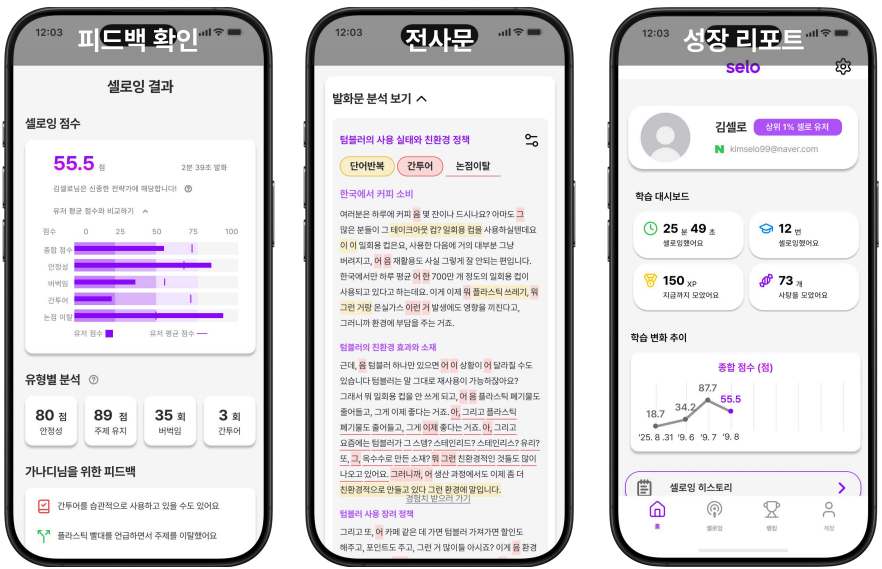
주제에 따라 자유롭게 발표를 시작합니다. 방, 사무실, 카페 등 어디서든 할 수 있습니다.

3) 실시간 발화 모니터링

발표 중 실시간으로 발화 속도와 톤을 모니터링하여 피드백을 제공합니다.



유형 분석 및 발화 대시보드



발화 데이터 확인 및 피드백

발화 중 측정된 데이터를 기반으로 피드백을 제공합니다.

전체 전사문 확인

전체 전사문을 보고, 발화 위치에 따른 피드백을 볼 수 있습니다.

성장 리포트

꾸준한 기록을 통해 성장하는 자신의 모습을 확인할 수 있습니다.

개선된 AI 목소리 들어보기

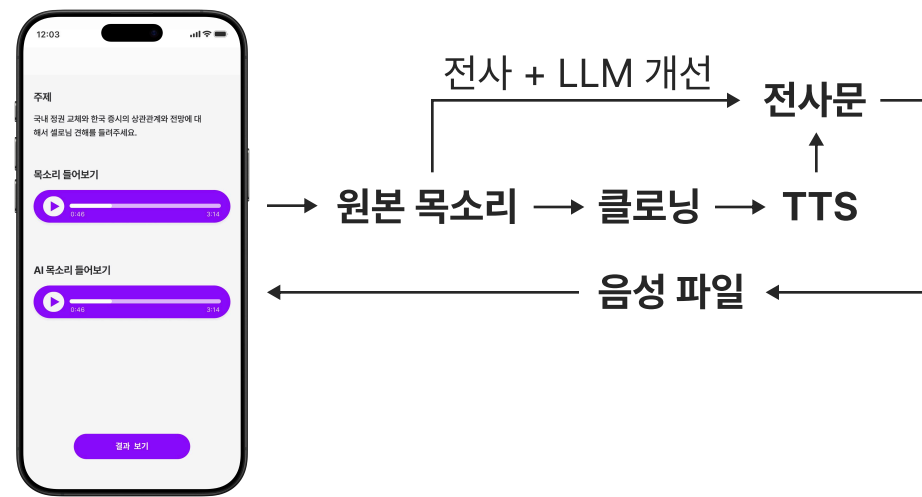
본인의 목소리로 정확한 발화를 생성하여 직관적인 발화 개선 기준을 제공합니다.

전사 STT - LLM 개선

발화를 실시간으로 전사하고 전사문을 LLM이 개선합니다.

CosyVoice3 - 보이스 클로닝

보이스 클로닝 후 만들어진 TTS 모델로 전사문을 읽습니다.



3. 기능 구현 과정

1) 상용 STT 모델의 한계

상황

Whisper나 CLOVA와 같은 범용 STT 모델은 **비유창성 정보를 제거하거나 보정**하는 경향이 있습니다. 일반적인 음성 인식에는 적합하지만, 비유창성을 분석해야 하는 **Selo**에서는 **핵심 평가 정보가 누락되어 서비스 정확도에 치명적인 한계**가 있었습니다.

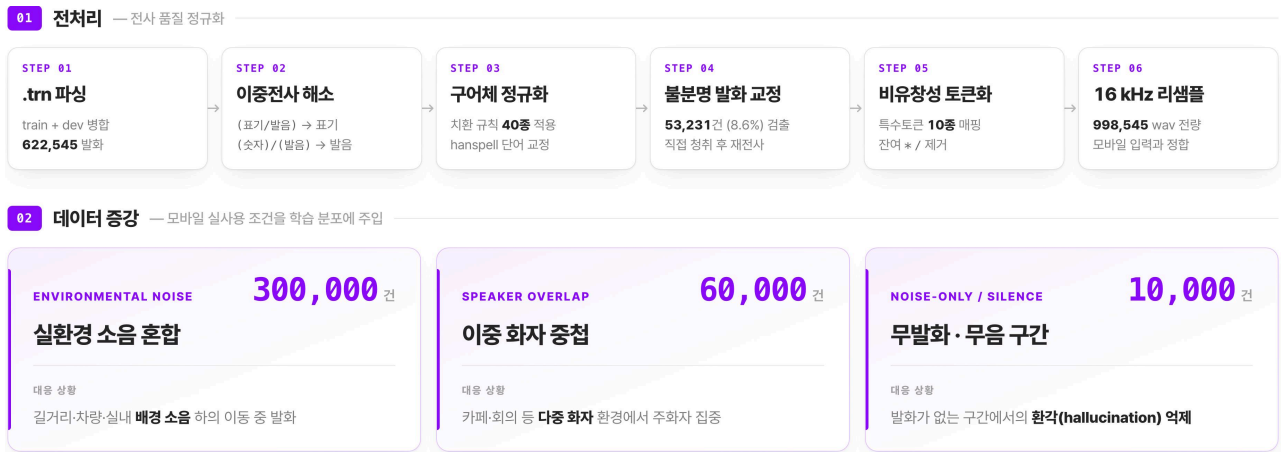
해결

a. 영어권 CrisperWhisper를 한국어의 언어학적 특성에 맞게 튜닝



b. 모바일 환경에서 잘 작동하도록 전처리 및 데이터 증강

고품질 녹음 환경에서 수집된 학습 데이터와 실제 모바일 사용 환경의 차이를 줄이기 위해, 노이즈·음질 저하 등 모바일 환경을 모사한 전처리와 데이터 증강을 적용했습니다.



회고

Token F1-score 0.92 달성

문제 해결을 위한 레퍼런스를 충분히 조사하고 학습하는 과정을 통해 목표를 달성했습니다. 이 과정에서 서비스와 환경적 특성을 고려하여 가설을 세우고 실험하는 것의 중요성을 배웠습니다.

2) 제한된 예산으로 고비용 GPU 인프라 운영이 어려운 문제

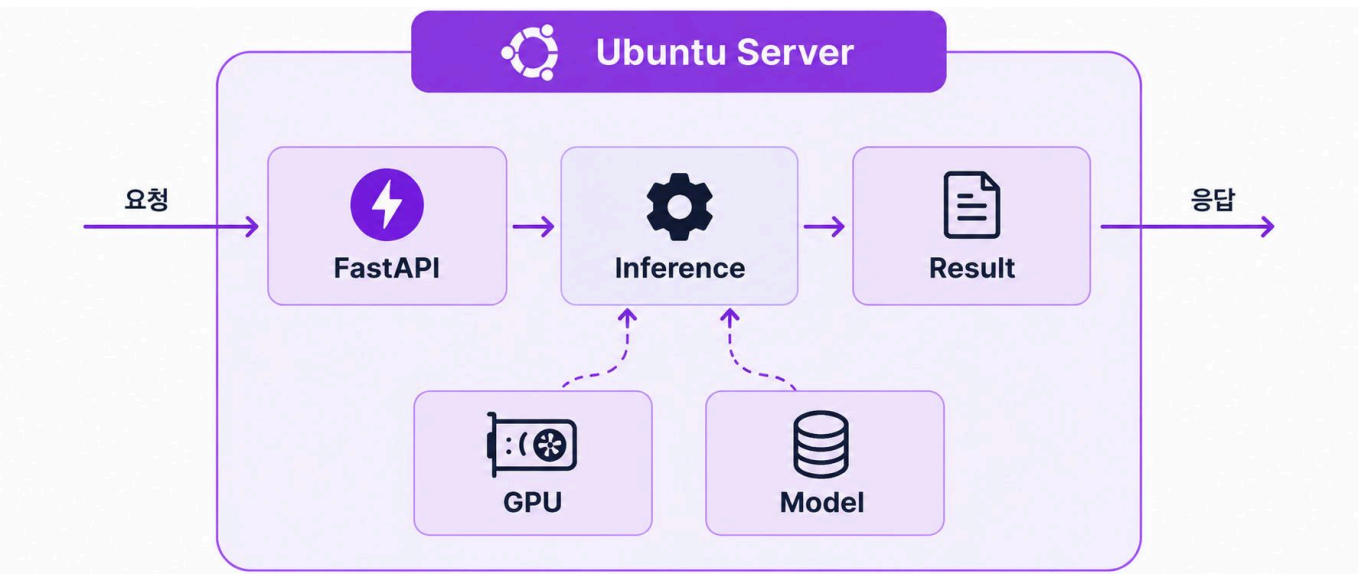
상황

클라우드 GPU 서버는 월 약 **\$472(g4dn.xlarge 기준)**의 비용이 발생해 지속적인 운영이 어려웠습니다. 이에 안정성이 다소 낮아지더라도 **비용을 최소화**하면서 **모델 추론을 수행할 수 있는 서버 환경**이 필요했습니다.

해결

a. 게이밍 PC를 Ubuntu GPU 서버로 개조

취미용 게이밍 PC를 포맷 후 Ubuntu를 설치하였습니다.



b. EC2 메인 서버와 GPU 추론 서버 연동

클라이언트 요청이 오면, 메인 서버가 온프레미스 GPU 서버에 추론을 요청하고 결과를 전달받도록 구성했습니다.



회고

가설을 세우고 수치로 검증하는 것

한정된 예산이라는 제약 속에서 **가용 자원을 활용**해 문제를 해결하고, 서비스에 연동하는 경험을 했습니다. 이를 통해 최적의 기술보다 **주어진 환경에 적합한 해결책**을 찾는 것이 **중요**하다는 것을 배웠습니다.

3) 개발 인력 부족으로 백엔드를 직접 학습해야 했던 문제

상황

프로젝트 당시 **개발 인력이 부족**해, AI와 기획을 담당하던 제가 **빠르게 백엔드를 학습해 개발에 투입**되어야 했습니다. 프론트엔드 개발자 역시 본업과 프로젝트를 병행하고 있어, 제한된 기간 내 모든 **개발을 단독으로 수행하기 어려운 상황**이었습니다.

해결

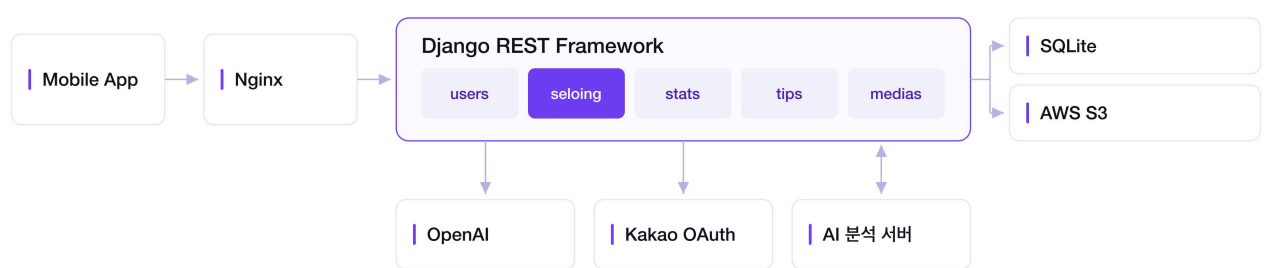
a. Django와 함께 Web 기초 학습

AI 부트캠프를 수료하며 익힌 Python으로 백엔드를 구축할 수 있는 Django를 빠르게 독학하였습니다.



b. 네트워크와 인프라 감각 훈련

웹 프레임워크로 백엔드를 구현하는 것과 이를 실제 사용자가 접근할 수 있는 서비스로 배포하는 것은 달랐습니다. 빠르게 서비스를 완성해야 했던 만큼 책으로만 학습하기보다 직접 구현하고 문제를 해결하는 과정을 통해 필요한 지식을 빠르게 익혔습니다.



회고

기술을 빠르게 습득하는 방법

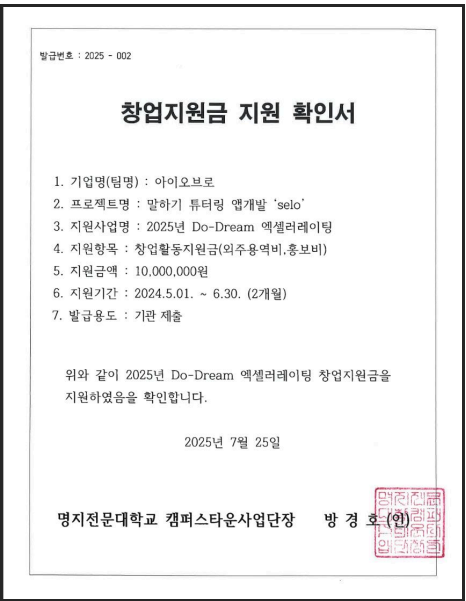
필요한 기술은 **빠르게 익혀 서비스에 적용**할 수 있다는 **자신감**을 얻었습니다. 동시에 Django를 배우며 OOP 개념을 이해하는 데 많은 시간이 걸렸고, 이를 통해 새로운 기술을 빠르게 익히기 위해서는 **탄탄한 기본기가 중요**하다는 것을 배웠습니다.

신촌 스타트업 박람회 참가 기업 선정부터 서울 캠퍼스타운 액셀러레이팅 선정까지

성과

액셀러레이팅 선정과 박람회 참가까지
서울 캠퍼스타운 지원과 신촌 스타트업 박람회 참여

액셀러레이팅 지원 확인서 | 박람회 참여 현장



산출물



GitHub - BE



GitHub - AI



Huggingface

공부한 것

SW 엔지니어링의 재미!
Computational Thinking의 즐거움
기초 알고리즘과 자료구조

데이터와 RDBMS
기능명세부터 ERD, SQL

Backend Framework
Django와 FastAPI를 통한 CRUD 구현

인프라 구성과 배포
Docker, Nginx, AWS 및 온프레미스 서버 구축

AI 활용

Hugging Face - Transformers
Whisper 모델을 받아오고 파인튜닝 파이프라인 구축

PyTorch를 통한 전처리 파이프라인 구축
PyTorch Audio를 이용한 오디오 전처리 및
Voice Cloning 모델 구축

프로젝트 소감 및 인사이트

- 서비스 간 흐름을 설계하고 유기적으로 연결하는 과정 경험
- ‘내가 만든 서비스를 누군가 실제로 사용한다’는 걸 알고, 개발의 즐거움을 알게 됨
- 개발자의 길을 선택하게 된 소중한 프로젝트였습니다.

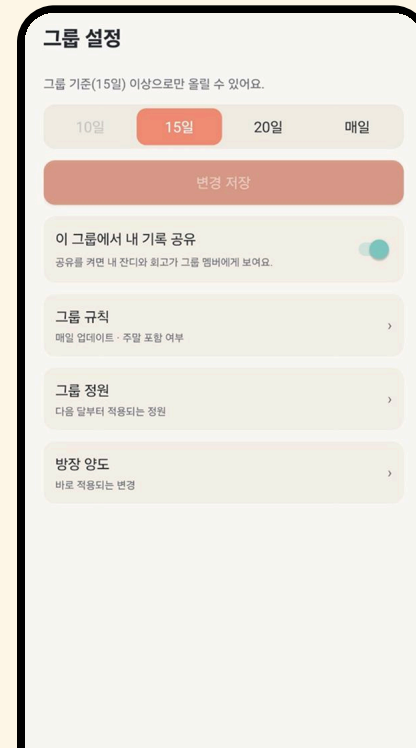
친구와 그룹을 맺어 목표를 공유하는 To-Do 서바이벌

열살방

AI 주도 개발 워크플로우를 테스트하기 위해
기획 - 설계 - 구현 - 테스트 전 과정을
BMAD-METHOD를 통해 제작했습니다.

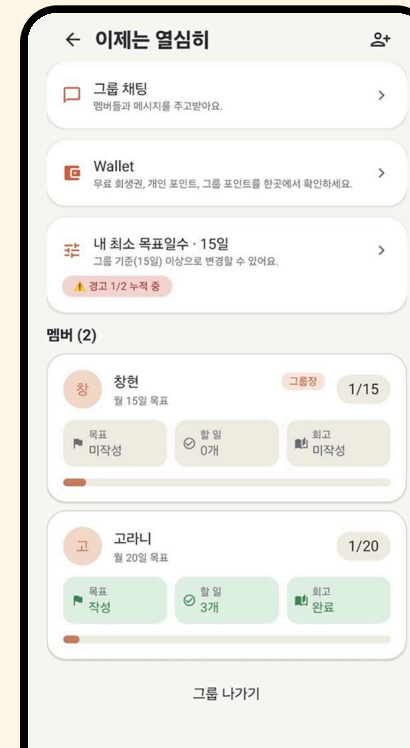
1) 친구와 그룹을 만들어 경쟁

그룹과 규칙을 만들고
친구와 함께 경쟁하세요!



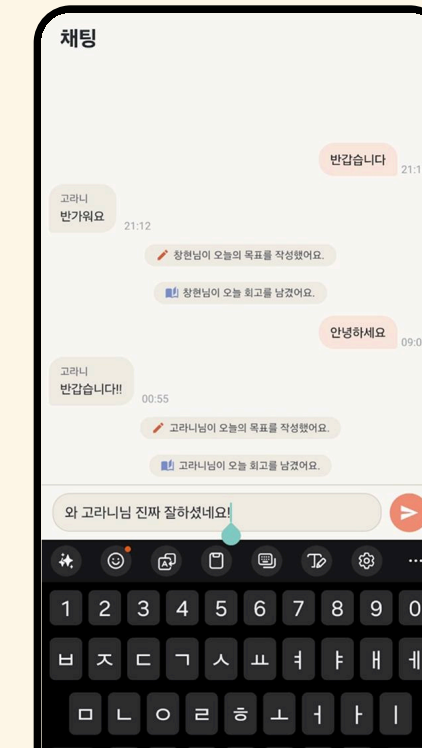
2) 목표 추적 대시보드

친구와 함께 목표를 달성하세요!
목표를 채우지 못하면 강제 퇴장입니다.



3) 채팅

달성도를 공유하고 대화하며
자극받으세요!



Quietity

과연결(過連結) 시대의 느슨한 연결 공간

우리의 뇌는 원시 시대에 머물러 있는데
정보화로 우리는 너무 많이 연결되어 있습니다.

Quietity는 과도하게 연결된 시대 속, 아무것도 요구받지 않는 공간입니다.
사용자는 자신의 기분을 남길 수도, 그저 머물 수도 있습니다.

기분을 선택하면, 당신의 기분이 하늘의 빛과 음악으로 천천히 번져갑니다.
타인의 기분엔 집중하지 않아도 됩니다.
혼자가 아니라는 것을 아는 게 더 중요합니다.



Live

앰비언트 뮤직

타이머

대륙별 기분 통계

평균 기분에 따른 변화

대나무숲

개발의 가장 매력적인 부분은
사람과 사람이 만나게 하고, 더 살기 좋은 세상을 만들 수 있게 해주는
가장 자유로운 도구라는 점입니다.

제가 가장 좋아하는 것은
0과 1로 Zero To One을 실현하는 것이라고 정의합니다.